

MICROSERVICES COURSE

Spring Boot & Spring Cloud — Professional Edition

Session 15

KUBERNETES FUNDAMENTALS

Dr. ElSayed Mohamed Elsayed Baladoh

Tech Lead · Ph.D. · sayedbaladoh@yahoo.com

MICROSERVICES COURSE

Spring Boot & Spring Cloud — Professional Edition

KUBERNETES FUNDAMENTALS

Beginner Student Tutorial

Prerequisite for Session 15 — Helm & Kubernetes Packaging

1. Why This Tutorial Exists

Session 14 introduced GitOps and Argo CD and therefore introduced Kubernetes as the deployment target. Session 15 then introduces Helm. Before students use Helm, they need a proper Kubernetes foundation. This tutorial fills that gap.

The course progression is: Docker → Kubernetes Fundamentals → Helm → GitOps/Argo CD + Helm → Advanced Kubernetes.

The existing Session 15 material assumes students already understand Deployments, Services, namespaces, image tags, and kubectl. [filecite](#)[turn2file2](#)[L151-L192](#)

2. Learning Objectives

- Explain what Kubernetes is and why Docker alone is not enough for production orchestration.
- Understand clusters, control planes, worker nodes, the Kubernetes API, and kubectl.
- Understand Kubernetes objects/resources and desired state.
- Read and write Kubernetes YAML manifests.
- Use namespaces, Pods, Deployments, ReplicaSets, Services, ConfigMaps, Secrets, and Ingress.
- Choose the correct Service type for a microservice.
- Understand labels/selectors and Service discovery.
- Use readiness, liveness, and startup probes with Spring Boot.
- Understand persistent storage at a beginner level.
- Recognize Jobs, CronJobs, StatefulSets, DaemonSets, and HPA.
- Run Kubernetes locally with Minikube, kind, or Docker Desktop.
- Understand managed Kubernetes in AWS, Azure, and Google Cloud.
- Deploy, scale, update, expose, inspect, and troubleshoot a Spring Boot service.
- Understand why Helm is introduced after raw Kubernetes YAML.

3. What Is Kubernetes?

Kubernetes is an open-source platform for managing containerized workloads and services using declarative configuration and automation. It continuously works to move the actual cluster state toward the desired state declared by the user. [cite](#)[turn0search0](#)[turn0search3](#)

Kubernetes provides service discovery/load balancing, automated rollouts and rollbacks, self-healing, scaling, resource scheduling, and configuration/secret management. [cite](#) [turn0search0](#)

4. Why Do We Need Kubernetes After Docker?

Docker packages and runs containers. Kubernetes manages many containerized workloads reliably across a cluster.

Docker:

build image → run container

Kubernetes:

desired state

↓

Kubernetes API

↓

controllers

↓

Pods

↓

containers

5. Docker vs Kubernetes

Docker/Compose is excellent for local container development. Kubernetes is designed to manage workloads across one or more nodes with scheduling, service discovery, scaling, rolling updates, and self-healing.

- Docker image = packaged application artifact.
- Container = running instance of an image.
- Pod = Kubernetes deployable unit containing one or more containers.
- Deployment = controller that manages stateless application replicas.
- Service = stable network endpoint for Pods.
- ConfigMap/Secret = externalized application configuration.
- Helm = later, packages and parameterizes these Kubernetes resources.
- Argo CD = later, reconciles Git desired state into Kubernetes.

6. Kubernetes Cluster

Kubernetes Cluster

|

+-- Control Plane

|

+-- kube-apiserver

|

+-- etcd

|

+-- scheduler

|

+-- controllers

|

+-- Worker Nodes

+-- kubelet

```
+++ container runtime
+++ Pods
```

A Kubernetes cluster consists of a control plane and one or more worker nodes. [cite](#)[turn0search5](#)

7. Control Plane Components

- kube-apiserver — exposes the Kubernetes HTTP API.
- etcd — stores Kubernetes API data and cluster state.
- kube-scheduler — selects suitable nodes for unscheduled Pods.
- kube-controller-manager — runs controllers that reconcile desired and actual state.
- cloud-controller-manager — optional integration with cloud infrastructure.

8. Worker Node Components

- kubelet — node agent responsible for making sure assigned Pods are running.
- Container runtime — runs containers.
- Networking components — implement the cluster networking model.

Students do not need to administer these components yet. They need to understand that the control plane manages desired state and worker nodes run workloads.

9. Kubernetes API and kubectl

Kubernetes is API-driven. kubectl is the command-line client that sends requests to the Kubernetes API.

```
kubectl
|
v
kube-apiserver
|
+--> stores/updates desired state
|
+--> controllers react
|
v
cluster resources
```

Kubernetes objects are persistent records of intent. Most have a desired spec and a system-maintained status. [cite](#)[turn0search3](#)

10. Kubernetes Objects / Resources

Important beginner resources:

- Namespace — logical grouping/isolation.
- Pod — smallest deployable compute unit.

- Deployment — stateless application rollout and replica management.
- ReplicaSet — maintains the desired number of matching Pods.
- Service — stable network endpoint for Pods.
- ConfigMap — non-sensitive configuration.
- Secret — sensitive data represented as a Kubernetes Secret object.
- Ingress — HTTP/HTTPS routing rules.
- PersistentVolume/PersistentVolumeClaim — persistent storage abstractions.
- Job/CronJob — one-time or scheduled work.
- StatefulSet — stateful workloads with stable identity/storage patterns.
- DaemonSet — workload intended to run on selected nodes.
- HorizontalPodAutoscaler — automatic replica scaling.

11. Desired State and Declarative Configuration

Kubernetes is primarily declarative. Instead of giving Kubernetes a long sequence of steps, you declare what you want.

Desired:

```
replicas: 3
image: product-service:1.0.0
```

Kubernetes:

```
continuously works toward that state
```

This concept becomes the foundation of GitOps in Session 14.

12. Kubernetes Manifest

A manifest is usually a YAML file describing a Kubernetes object.

```
apiVersion:
kind:
metadata:
spec:
```

- apiVersion — API version.
- kind — object type.
- metadata — name, namespace, labels, annotations, etc.
- spec — desired configuration.
- status — normally maintained by Kubernetes rather than written by students.

Kubernetes documentation describes manifests as the usual YAML/JSON representation used to express objects. [cite](#) [turn0search3](#)

13. First Pod Manifest

```
apiVersion: v1
kind: Pod
```

```
metadata:
  name: hello-pod
spec:
  containers:
  - name: hello
    image: nginx:1.27
    ports:
    - containerPort: 80
```

This creates one Pod. It is useful for learning, but a production Spring Boot service should normally be managed by a Deployment.

14. Namespaces

Namespaces group resources logically and allow policies and permissions to be applied by namespace.

```
kubectl create namespace ecommerce
kubectl get namespaces
kubectl get pods -n ecommerce
kubectl get pods -A
```

A course environment may use ecommerce-dev, ecommerce, and ecommerce-prod. A namespace is useful isolation, but it is not by itself a complete security boundary.

15. Labels and Selectors

Labels identify resources. Selectors find matching resources.

```
metadata:
  labels:
    app: product-service

selector:
  app: product-service
```

A Service uses selectors to find Pods. A Deployment uses selectors to manage the correct Pod set. A mismatched selector is one of the most common beginner errors.

16. Pod

A Pod is the smallest deployable compute unit in Kubernetes. Containers inside a Pod share the Pod network namespace and can share volumes.

```
Pod
+-- product-service container
+-- optional sidecar containers
+-- shared network identity
```

Pods are ephemeral. Do not design clients around permanent Pod IP addresses.

17. Deployment

A Deployment is the normal Kubernetes workload abstraction for a stateless Spring Boot microservice.

Deployment

|

v

ReplicaSet

|

+--> Pod

+--> Pod

+--> Pod

The Deployment provides declarative replicas, rolling updates, and rollback capabilities.

18. Product Service Deployment

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: product-service
  namespace: ecommerce
spec:
  replicas: 2
  selector:
    matchLabels:
      app: product-service
  template:
    metadata:
      labels:
        app: product-service
    spec:
      containers:
        - name: product-service
          image: ghcr.io/OWNER/product-service:1.0.0
          ports:
            - containerPort: 8080
```

19. Deployment Commands

```
kubectl apply -f deployment.yaml
kubectl get deployment -n ecommerce
kubectl get replicaset -n ecommerce
kubectl get pods -n ecommerce
kubectl describe deployment product-service -n ecommerce
kubectl rollout status deployment/product-service -n ecommerce
kubectl rollout history deployment/product-service -n ecommerce
```

20. Scaling

```
kubectl scale deployment/product-service --replicas=4 -n ecommerce
kubectl get pods -n ecommerce
```

The Deployment controller creates or removes Pods until the desired number is reached.

21. Rolling Update and Rollback

```
kubectl set image deployment/product-service product-service=ghcr.io/OWNER/product-service:1.1.0 -n ecommerce
```

```
kubectl rollout status deployment/product-service -n ecommerce
```

```
kubectl rollout undo deployment/product-service -n ecommerce
```

Kubernetes performs the rollout; Helm later provides a higher-level release-management workflow.

22. Service — Why It Exists

Pods are replaceable. Their IP addresses can change. A Service gives a stable endpoint and selects the correct Pods.

```
Client
|
v
Service: product-service:8080
|
+--> Pod A
+--> Pod B
+--> Pod C
```

Kubernetes defines a Service as a method for exposing a network application running in one or more Pods. [cite](#) [turn0search11](#)

23. ClusterIP Service

```
apiVersion: v1
kind: Service
metadata:
  name: product-service
  namespace: ecommerce
spec:
  selector:
    app: product-service
  ports:
    - port: 8080
      targetPort: 8080
  type: ClusterIP
```

Inside the cluster, another service can call `http://product-service:8080`.

24. port vs targetPort

- port — port exposed by the Service.
- targetPort — port on the selected Pod/container.
- nodePort — port exposed on nodes when using NodePort.

Service port 8080
↓
Pod targetPort 8080
↓
Spring Boot server.port=8080

25. Service Types

- ClusterIP — internal access; default and normal choice for internal microservices.
- NodePort — exposes a port on each node; useful for simple local/testing scenarios.
- LoadBalancer — requests an external load-balancing endpoint from supporting infrastructure, common in cloud.
- ExternalName — DNS alias to an external name for special integration scenarios.

For this course, Product, Inventory, Order, Payment, etc. should normally use ClusterIP. The edge/API gateway should be the externally exposed entry point.

26. Which Service Type Should I Use?

Internal microservice → ClusterIP

Local simple external test → NodePort or port-forward/minikube service

Cloud external endpoint → LoadBalancer

HTTP routing for multiple Services → Ingress/Gateway API + controller/implementation

Do not expose every microservice publicly just because it has an HTTP endpoint.

27. Headless Service — Awareness

spec:

clusterIP: None

A headless Service is useful for direct Pod discovery, especially in stateful systems. It is not needed for the basic Product Service lab.

28. Kubernetes Service Discovery

```
order-service
|
| http://product-service:8080
v
product-service Service
|
```

```
+--> Product Pod
+--> Product Pod
```

Kubernetes DNS provides stable names for Services. This removes the need for applications to know Pod IP addresses.

29. ConfigMap

ConfigMap stores non-sensitive configuration separately from the image.

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: product-service-config
  namespace: ecommerce
data:
  SPRING_PROFILES_ACTIVE: "kubernetes"
  LOG_LEVEL: "INFO"
  APP_NAME: "product-service"
```

A ConfigMap can be injected as environment variables or mounted as files.

30. ConfigMap as Environment Variables

```
envFrom:
- configMapRef:
  name: product-service-config
```

Use ConfigMap for non-sensitive values such as application profile, log level, feature configuration, and non-secret endpoints.

31. Secret

Kubernetes Secrets are intended for sensitive values such as passwords, tokens, and keys.

```
apiVersion: v1
kind: Secret
metadata:
  name: product-service-secret
  namespace: ecommerce
type: Opaque
stringData:
  DB_USERNAME: product
  DB_PASSWORD: change-me
```

Important: Secret does not mean 'automatically fully secure'. Access control and appropriate encryption-at-rest/secret-management practices are still required. Never commit real production secrets to Git.

32. Secret as Environment Variables

```
env:
  - name: DB_PASSWORD
    valueFrom:
      secretKeyRef:
        name: product-service-secret
        key: DB_PASSWORD
```

For production, a dedicated secret manager or cloud secret service may be preferable.

33. ConfigMap vs Secret

- ConfigMap = non-sensitive configuration.
- Secret = sensitive configuration/data.
- Both can be injected as environment variables.
- Both can be mounted as files.
- Neither should be confused with application code inside the Docker image.

34. Spring Boot Configuration Model

Docker Image

- +-- Java application
- +-- dependencies

ConfigMap

- +-- non-sensitive environment configuration

Secret

- +-- passwords/tokens/keys

Deployment

- +-- connects the application to configuration

35. Health Probes

Probe	Question	Purpose
Startup	Has the application finished starting?	Protect slow-starting applications
Readiness	Can this Pod receive traffic?	Control Service traffic
Liveness	Is the application still alive?	Restart a stuck container

A Pod can be Running but not Ready. That distinction is critical for reliable Services.

36. Spring Boot Probe Example

```
management:
  endpoint:
    health:
      probes:
        enabled: true
```

```

readinessProbe:
  httpGet:
    path: /actuator/health/readiness
    port: 8080
  initialDelaySeconds: 10
  periodSeconds: 5

```

```

livenessProbe:
  httpGet:
    path: /actuator/health/liveness
    port: 8080
  initialDelaySeconds: 30
  periodSeconds: 10

```

Tune timings using real startup/health behavior; do not copy arbitrary production values.

37. Ingress

Ingress is a Kubernetes HTTP/HTTPS routing API object. It can route requests by host/path to Services. An Ingress Controller is required to implement the routing.

```

apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: ecommerce-ingress
  namespace: ecommerce
spec:
  rules:
    - host: api.example.local
      http:
        paths:
          - path: /products
            pathType: Prefix
            backend:
              service:
                name: product-service
                port:
                  number: 8080

```

Ingress does not by itself create an external load balancer or controller. Kubernetes currently recommends Gateway API for new development; Ingress remains stable but its API is frozen.

□ cite □ turn0search9 □

38. Ingress vs Spring Cloud Gateway

- Spring Cloud Gateway is an application/API gateway used for application-level routing and filters.
- Kubernetes Ingress is a Kubernetes traffic-routing API object.
- An Ingress Controller implements Ingress behavior.

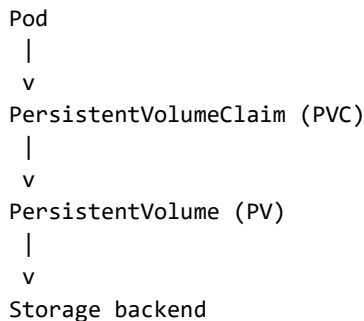
- Spring Cloud Gateway may remain the main application gateway while an Ingress/Gateway API layer provides cluster entry.

39. Gateway API — Awareness

Gateway API is the modern Kubernetes direction for expressive traffic management. Students should know it exists and that Kubernetes recommends it for new development, but Ingress is still important because it is widely deployed and remains stable. [cite turn0search9](#)

40. Persistent Storage

A container filesystem should not be treated as durable database storage. Kubernetes provides persistent storage abstractions.



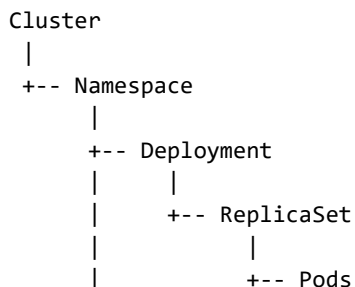
- PV — storage resource available to the cluster.
- PVC — application request for storage.
- StorageClass — describes dynamic storage provisioning.

41. Jobs, CronJobs, StatefulSets, DaemonSets

- Job — run work until completion, e.g. one-time migration.
- CronJob — create Jobs on a schedule.
- StatefulSet — stateful workloads needing stable identity/storage patterns.
- DaemonSet — run a Pod on each eligible node, common for node agents.

These are awareness topics now; the course can revisit them when needed.

42. Kubernetes Resource Map



```
|
+-- Service → selects Pods
+-- ConfigMap
+-- Secret
+-- Ingress / Gateway resources
+-- PVC
+-- HPA (later)
```

43. Request Flow

```
Client
|
v
Load Balancer / Ingress / Gateway
|
v
Service
|
+--> Pod A
+--> Pod B
+--> Pod C
    |
    v
    Spring Boot
```

44. Docker Image to Kubernetes

```
Spring Boot source
|
v
Maven tests/build
|
v
Docker/OCI image
|
v
Container Registry
|
v
Deployment
|
v
Pods
|
v
Service
|
v
Ingress/Gateway/LoadBalancer
```

45. Essential kubectl Commands

```
kubectl version --client
kubectl cluster-info
kubectl get nodes
kubectl get namespaces
kubectl get pods -A
kubectl get all -n ecommerce
kubectl describe pod <pod> -n ecommerce
kubectl logs <pod> -n ecommerce
kubectl exec -it <pod> -n ecommerce -- sh
kubectl apply -f file.yaml
kubectl delete -f file.yaml
kubectl get svc -n ecommerce
kubectl get ingress -n ecommerce
kubectl get events -n ecommerce --sort-by=.lastTimestamp
```

46. Troubleshooting: The Golden Sequence

1. Which cluster?
 kubectl config current-context
2. Are nodes healthy?
 kubectl get nodes
3. What is the Pod state?
 kubectl get pods -n ecommerce
4. Why?
 kubectl describe pod <pod> -n ecommerce
5. What does the app say?
 kubectl logs <pod> -n ecommerce
6. What do cluster events say?
 kubectl get events -n ecommerce --sort-by=.lastTimestamp
7. Is the Service selecting Pods?
 kubectl get endpointslice -n ecommerce
8. Are Pods Ready?
 kubectl describe pod <pod> -n ecommerce

47. Common Pod Problems

- Pending — scheduling, resource, storage, or admission problem.
- CrashLoopBackOff — application starts then repeatedly crashes.
- ImagePullBackOff — image cannot be pulled.
- Running but not Ready — readiness probe/configuration problem.
- Service has no response — selector, ports, readiness, or application-listening-port problem.
- OOMKilled — container exceeded memory limit or has a memory problem.

48. Local Kubernetes Options

The official Kubernetes learning guidance recommends local environments such as Minikube and kind for learning and experimentation. [cite](#)[turn0search4](#)

- Minikube — beginner-friendly local Kubernetes.
- kind — Kubernetes nodes running as Docker/Podman containers.
- Docker Desktop Kubernetes — convenient if Docker Desktop is already installed.
- Online playground — useful for quick experiments.
- kubeadm — advanced cluster creation/administration; not the first learning environment.

49. Recommended Course Setup

Because students already use Docker Desktop, use either Minikube with the Docker driver or kind. Minikube is a good primary beginner walkthrough; kind is an excellent alternative.

The official Kubernetes tooling documentation lists both Minikube and kind as local options. [cite](#)[turn0search1](#)[turn0search6](#)

50. Minikube — Start a Local Cluster

```
minikube start --driver=docker
minikube status
kubectl get nodes
kubectl cluster-info
```

The official Minikube tutorial uses minikube start and minikube status to create and verify a local cluster. [cite](#)[turn0search7](#)

51. First Local Deployment

```
kubectl create namespace ecommerce

kubectl create deployment hello --image=nginx:1.27 -n ecommerce

kubectl get deployment -n ecommerce
kubectl get pods -n ecommerce
```

52. Expose and Test Locally

```
kubectl expose deployment hello --type=ClusterIP --port=80 -n ecommerce

kubectl get svc -n ecommerce

kubectl port-forward service/hello 8080:80 -n ecommerce
```

Open <http://localhost:8080>. Port-forward is a development/testing technique, not the normal production exposure model.

53. Minikube Service Helper

```
minikube service hello -n ecommerce
```

This is a convenient local-development mechanism for reaching a Service.

54. Minikube Stop/Delete

```
minikube stop
minikube start
minikube delete
```

55. kind — Alternative

```
kind create cluster --name microservices
kubectl get nodes
kubectl cluster-info
```

```
kind delete cluster --name microservices
```

kind is designed to run Kubernetes nodes in containers and is useful for development/testing.

□ cite □ turn0search6 □

56. Kubernetes Contexts

kubectl may be configured to access multiple clusters. Always know which context you are using.

```
kubectl config get-contexts
kubectl config current-context
kubectl config use-context <context-name>
```

57. Cloud Kubernetes

Students do not need a cloud account to learn Kubernetes fundamentals. After learning locally, the same Kubernetes concepts can be used with managed cloud services.

- AWS — Amazon EKS.
- Microsoft Azure — AKS.
- Google Cloud — GKE.

Managed Kubernetes reduces the amount of control-plane infrastructure the customer must operate, although workload, networking, security, scaling, storage, and application operations still require management.

58. Cloud Deployment Mental Model

```
GitHub
|
v
```

```

CI
  +--> Test
  +--> Build
  +--> Push image
  |
  v
Registry
  |
  v
GitOps Repository
  |
  v
Argo CD
  |
  v
Managed Kubernetes
  |
  +--> Deployment
  +--> Service
  +--> ConfigMap
  +--> Secret
  +--> Ingress/Gateway
  +--> HPA

```

59. Product Service — Complete Raw YAML Example

```

# namespace.yaml
apiVersion: v1
kind: Namespace
metadata:
  name: ecommerce

---

# configmap.yaml
apiVersion: v1
kind: ConfigMap
metadata:
  name: product-service-config
  namespace: ecommerce
data:
  SPRING_PROFILES_ACTIVE: kubernetes
  LOG_LEVEL: INFO

---

# deployment.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: product-service
  namespace: ecommerce
spec:
  replicas: 2

```

```

selector:
  matchLabels:
    app: product-service
template:
  metadata:
    labels:
      app: product-service
  spec:
    containers:
      - name: product-service
        image: ghcr.io/OWNER/product-service:1.0.0
        ports:
          - containerPort: 8080
        envFrom:
          - configMapRef:
              name: product-service-config
---
# service.yaml
apiVersion: v1
kind: Service
metadata:
  name: product-service
  namespace: ecommerce
spec:
  selector:
    app: product-service
  ports:
    - port: 8080
      targetPort: 8080
  type: ClusterIP

```

60. Apply the Raw YAML

```

kubectl apply -f namespace.yaml
kubectl apply -f configmap.yaml
kubectl apply -f deployment.yaml
kubectl apply -f service.yaml

```

```

kubectl get all -n ecommerce

```

61. Verify the Product Service

```

kubectl get pods -n ecommerce
kubectl describe pod <pod> -n ecommerce
kubectl logs <pod> -n ecommerce
kubectl get svc -n ecommerce
kubectl get endpointslice -n ecommerce

```

62. Test From Inside the Cluster

```
kubectl run curl-test --image=curlimages/curl --rm -it -n ecommerce -- sh
```

```
curl http://product-service:8080/actuator/health
```

The exact Actuator endpoint depends on the Spring Boot configuration used by the project.

63. Demonstrate Self-Healing

```
kubectl get pods -n ecommerce
kubectl delete pod <pod> -n ecommerce
kubectl get pods -n ecommerce -w
```

The Deployment/ReplicaSet creates a replacement because the desired replica count is still two.

64. Demonstrate Scaling

```
kubectl scale deployment/product-service --replicas=4 -n ecommerce
kubectl get pods -n ecommerce
```

65. Demonstrate Rolling Update

```
kubectl set image deployment/product-service product-service=ghcr.io/OWNER/product-
service:1.1.0 -n ecommerce
```

```
kubectl rollout status deployment/product-service -n ecommerce
```

66. Why We Learn Raw YAML Before Helm

If students start directly with Helm, the templates can look like magic. The correct progression is:

```
Raw Kubernetes YAML
  |
  v
Understand Deployment/Service/ConfigMap/Secret/etc.
  |
  v
Repeated YAML becomes obvious
  |
  v
Helm values + templates
  |
  v
Rendered Kubernetes YAML
```

Session 15 explicitly uses this mental model: values.yaml → templates → rendered Kubernetes YAML → Kubernetes. [filecite](#) [turn2file9](#) [L764-L833](#)

67. Kubernetes vs Docker Compose

- Docker Compose service → usually Deployment + Service in Kubernetes.
- Docker Compose container → container inside a Pod.
- Compose scale → Deployment replicas.
- Compose environment variables → ConfigMap/Secret/env.
- Compose ports → Service.
- Compose volumes → Kubernetes volumes/PVCs.
- Compose network → Kubernetes networking + Services + DNS.
- docker compose up → kubectl apply / Helm / GitOps.
- depends_on does not have a direct Kubernetes equivalent; use readiness/health and correct application behavior.

68. What Kubernetes Does NOT Do

- It does not build Spring Boot source code.
- It does not replace Maven.
- It does not replace container images.
- It does not automatically provide your database, Redis, Kafka, or monitoring stack.
- It does not make application code correct.
- It does not remove security responsibilities.
- It does not automatically make every stateful application horizontally scalable.
- It does not replace CI or GitOps.

The official Kubernetes overview explicitly notes that Kubernetes does not build application source code and does not prescribe a complete CI/CD system or application middleware stack.

[cite](#) [turn0search0](#)

69. Beginner Lab — Kubernetes From Zero

1. Install kubectl and Minikube.
2. Start Minikube with Docker driver.
3. Verify node status.
4. Create ecommerce namespace.
5. Create an NGINX Deployment.
6. Scale it to three replicas.
7. Create a ClusterIP Service.
8. Access it with kubectl port-forward.
9. Delete a Pod and observe self-healing.
10. Update the image and observe a rolling update.
11. Repeat the exercise with Product Service.
12. Add a ConfigMap.
13. Add a development-only Secret.
14. Inspect logs, describe output, and events.
15. Explain every YAML field before moving to Helm.

70. Common Beginner Mistakes

- Calling Pod IPs directly.
- Mismatching Deployment selectors and Pod labels.
- Confusing Service port and targetPort.
- Using NodePort for every microservice.
- Assuming ClusterIP is publicly reachable.
- Creating Ingress without an Ingress Controller.
- Putting passwords in ConfigMaps.
- Committing real Secrets to Git.
- Using latest everywhere.
- Inspecting the wrong namespace.
- Using the wrong kubectl context.
- Assuming Running means Ready.
- Treating a Pod as permanent.
- Deleting everything instead of reading describe/logs/events.

71. Debugging Checklist

1. `kubectl config current-context`
2. `kubectl get nodes`
3. `kubectl get pods -n ecommerce`
4. `kubectl describe pod <pod> -n ecommerce`
5. `kubectl logs <pod> -n ecommerce`
6. `kubectl get events -n ecommerce --sort-by=.lastTimestamp`
7. `kubectl get svc -n ecommerce`
8. `kubectl get endpointslice -n ecommerce`

72. Quick Quiz

1. What is Kubernetes?

Answer: A platform for managing containerized workloads using declarative configuration and automation.

2. Why is Docker not enough for a production microservices cluster?

Answer: Docker runs containers; Kubernetes manages many workloads with scheduling, service discovery, scaling, self-healing, and reconciliation.

3. What is a Pod?

Answer: The smallest deployable compute unit in Kubernetes.

4. Why not use Pod IPs as stable addresses?

Answer: Pods are ephemeral and can be replaced.

5. What does a Deployment manage?

Answer: Stateless application replicas and rolling updates through ReplicaSets and Pods.

6. What is a Service?

Answer: A stable network endpoint that selects a set of Pods.

7. Which Service type is normally used for internal microservices?

Answer: ClusterIP.

8. What is targetPort?

Answer: The port on the selected Pod/container receiving Service traffic.

9. What is a ConfigMap?

Answer: Non-sensitive configuration data.

10. What is a Secret?

Answer: A Kubernetes object intended for sensitive data such as passwords and tokens.

11. Does Secret automatically mean the data is completely secure?

Answer: No; RBAC and appropriate encryption/secret-management practices are still required.

12. What is readiness?

Answer: Whether a Pod is ready to receive traffic.

13. What is liveness?

Answer: Whether a running container should be considered alive and restarted if necessary.

14. What is Ingress?

Answer: A Kubernetes HTTP/HTTPS routing API object implemented by an Ingress Controller.

15. What is the modern direction for new Kubernetes traffic management?

Answer: Gateway API.

16. What is kubectl?

Answer: The Kubernetes command-line client.

17. What is Minikube?

Answer: A local Kubernetes environment for learning/development.

18. What is kind?

Answer: A tool that runs Kubernetes nodes as containers.

19. Why learn raw YAML before Helm?

Answer: To understand the Kubernetes resources that Helm will later package and template.

73. Student Readiness Checklist Before Session 15

- I can explain Kubernetes in my own words.
- I understand control plane vs worker node.
- I know what kubectl does.
- I can read apiVersion, kind, metadata, and spec.
- I can create and use a namespace.
- I understand Pod vs Deployment.
- I understand Deployment → ReplicaSet → Pod.
- I understand labels and selectors.
- I can create and scale a Deployment.
- I understand rolling updates and rollback.
- I understand why a Service is required.
- I know when to use ClusterIP, NodePort, and LoadBalancer.
- I understand port vs targetPort.
- I can use ConfigMap for non-sensitive configuration.
- I understand Secret and its limitations.
- I understand readiness, liveness, and startup probes.
- I know what Ingress and an Ingress Controller are.
- I know Gateway API is the modern direction.
- I understand basic persistent storage concepts.
- I can run Kubernetes locally.
- I can troubleshoot with get, describe, logs, and events.
- I understand Docker Image → Deployment → Pod → Service.

74. How This Leads Into Session 15

A real microservice may require Deployment, Service, ConfigMap, Secret, Ingress, HPA, and other resources. Repeating similar YAML for product-service, inventory-service, order-service, payment-service, and multiple environments quickly becomes difficult.

Session 15 solves this with Helm: charts, values, templates, releases, environment-specific configuration, and Helm integration with Argo CD. The existing Session 15 material explicitly identifies repeated Deployment, Service, ConfigMap, Ingress, and HPA YAML as the problem Helm addresses. [filecite](#) [turn2file4](#) [L306-L318](#)

75. Full Course Mental Model

Source Code

|

v

Maven + Tests

Kubernetes Objects: <https://kubernetes.io/docs/concepts/overview/working-with-objects/>

Kubernetes Components: <https://kubernetes.io/docs/concepts/overview/components/>

Services: <https://kubernetes.io/docs/concepts/services-networking/service/>

Ingress: <https://kubernetes.io/docs/concepts/services-networking/ingress/>

Install Tools: <https://kubernetes.io/docs/tasks/tools/>

Learning Environment: <https://kubernetes.io/docs/setup/learning-environment/>

Minikube: <https://kubernetes.io/docs/tutorials/kubernetes-basics/create-cluster/cluster-intro/>

Hello Minikube: <https://kubernetes.io/docs/tutorials/hello-minikube/>

78. Final Takeaway

Docker:

"Here is my application image."

Kubernetes:

"Keep my application running in the desired state."

Deployment:

"Keep N replicas."

Service:

"Give these Pods a stable endpoint."

ConfigMap:

"Here is non-sensitive configuration."

Secret:

"Here is sensitive configuration."

Ingress/Gateway:

"Route external traffic."

kubectl:

"Let me inspect/control the Kubernetes API."

Helm:

"Package and parameterize these Kubernetes resources."

Argo CD:

"Keep Kubernetes synchronized with Git."

After this tutorial, students should be able to start Session 15 without treating Helm as magic. Helm becomes the next layer above Kubernetes rather than a replacement for Kubernetes.